

Assignment 3 – Architecture Description

BME.70034: Empirical Asset Pricing

AhRam Cho
20253751

June 21, 2026

1 Repository

URL: <https://github.com/ahram-lara-cho/Empirical-Asset-Pricing-Assignment-2>

The repository contains a full Python pipeline for replicating the required empirical outputs from Gu, Kelly, and Xiu (2020): Table 1, Figure 4, and Figure 9. The code is organized as six sequential scripts under `src/`, with a single top-level driver script, `main.py`, that runs the complete pipeline end-to-end.

To reproduce the results, clone the repository, install the dependencies, place the required external macro predictor file in `data/raw/`, and execute the master script.

```
git clone https://github.com/ahram-lara-cho/Empirical-Asset-Pricing-Assignment-2
cd Empirical-Asset-Pricing-Assignment-2
```

```
conda create -n gkx python=3.11 -y
conda activate gkx
pip install -r requirements.txt
```

```
python main.py
```

WRDS credentials are required for the CRSP data download step and for the S&P 500 / risk-free benchmark used in Figure 9. The repository does not include WRDS-derived raw or processed data.

The macro predictor file from Amit Goyal must be downloaded manually and saved as:

```
data/raw/PredictorData.xlsx
```

2 Project Structure

The project is organized as a modular pipeline. Each script is responsible for one stage of the empirical workflow.

```

gkx_replication/
|
|-- main.py                # Master script: runs steps 01--06
|-- requirements.txt        # Python dependencies
|-- README.md              # Reproduction instructions
|-- .gitignore             # Excludes raw data, panels, and large outputs
|
|-- src/
|   |-- 01_build_panel.py   # Build CRSP/OSAP/macro prediction panel
|   |-- 02_train_simple_models.py # OLS+H, OLS-3+H, PCR, ENet+H
|   |-- 03_train_ml_models.py # RF, NN2, NN4
|   |-- 04_make_figure4.py  # Variable importance heatmaps
|   |-- 05_make_figure9.py  # Long-short portfolio cumulative returns
|   |-- 06_make_final1_table.py # Final Table 1 tables and figures
|
|-- data/
|   |-- raw/
|   |   |-- PredictorData.xlsx # User-provided macro predictors
|   |
|   |-- processed/
|   |   |-- char_cols.csv      # Generated feature metadata
|   |   |-- macro_cols.csv    # Generated feature metadata
|   |   |-- interaction_cols.csv # Generated feature metadata
|   |   |-- panel_1971_2025.parquet # Generated, not committed to GitHub
|   |
|-- outputs/
|   |-- predictions_simple.parquet # Generated, not committed to GitHub
|   |-- predictions_ml.parquet     # Generated, not committed to GitHub
|   |-- table1_final_set1.csv
|   |-- table1_final_set2.csv
|   |-- table1_final_display_set1.csv
|   |-- table1_final_display_set2.csv
|   |-- figure_table1_oos_r2_set1.png
|   |-- figure_table1_oos_r2_set2.png
|   |-- figure4_importance_set1.png
|   |-- figure4_importance_set2.png
|   |-- figure9_cumret_set1.png
|   |-- figure9_cumret_set2.png

```

The files `char_cols.csv`, `macro_cols.csv`, and `interaction_cols.csv` are small metadata

files listing the characteristics, macro predictors, and interaction terms used in the analysis. They are included for transparency and are regenerated automatically by `01_build_panel.py`. Large WRDS-derived data files and model prediction files are excluded from the GitHub repository.

3 Master Script

The top-level driver, `main.py`, sequentially executes all six steps of the project:

- (1) Build the feature panel.
- (2) Train simple linear and regularized models.
- (3) Train random forest and neural network models.
- (4) Construct Figure 4 variable importance heatmaps.
- (5) Construct Figure 9 long-short portfolio cumulative return plots.
- (6) Compile final Table 1 tables and Table 1 figures.

The master script uses Python’s `subprocess` module to execute each script in order using the same Python executable as the active environment. Logs for each step are written to:

`outputs/run_logs/`

The full pipeline can be run as:

```
python main.py
```

If earlier steps have already been completed, selected parts of the pipeline can be rerun. For example, to regenerate only figures and tables:

```
python main.py --from-step 4
```

To regenerate only Table 1:

```
python main.py --from-step 6 --to-step 6
```

4 Data Construction

4.1 CRSP Monthly Returns

The script `01_build_panel.py` downloads monthly U.S. stock returns from WRDS CRSP using `crsp.msf` joined with `crsp.msnames`. The sample begins in January 1971. The assignment target end date is 2025; however, at the time of execution, the available CRSP monthly stock file returned observations only through December 2024. Because the prediction target is the next-month return, the final usable feature month is November 2024.

The stock universe is restricted to NYSE common stocks using the following filters:

- Share codes `shrcd` in $\{10, 11\}$, corresponding to ordinary common stocks.
- Exchange code `exchcd` = 1, corresponding to NYSE.
- Non-missing monthly return.
- Non-missing price and shares outstanding for market equity construction.

Market equity is computed as:

$$ME_{i,t} = |PRC_{i,t}| \times SHROUT_{i,t}.$$

The next-month prediction target is constructed by aligning each stock-month observation at month t with its return in month $t + 1$:

$$r_{i,t}^{fwd} = r_{i,t+1}.$$

This ensures that all predictors dated month t are used only to predict future returns.

4.2 Firm Characteristics

Firm characteristics are downloaded from Open Source Asset Pricing using the `openassetpricing` Python package. The code supports either a selected signal list or a broader signal set. In the implemented version, CRSP-based size is added directly from market equity:

$$Size_{i,t} = \log(ME_{i,t}).$$

All firm characteristics are rank-normalized cross-sectionally within each month:

$$x_{i,t}^{rank} = 2 \times \text{rankpct}(x_{i,t}) - 1.$$

This maps each characteristic to the interval $[-1, 1]$. Missing characteristic values are filled with zero, which corresponds approximately to the cross-sectional median after rank-normalization.

4.3 Macroeconomic Predictors

Macroeconomic predictors are constructed from Amit Goyal's monthly predictor file, `PredictorData.xlsx`. The script reads the monthly sheet and constructs the following macro variables:

- Dividend-price ratio: `dp_macro`
- Earnings-price ratio: `ep_macro`
- Book-to-market: `bm_macro`
- Net equity issuance: `ntis_macro`

- Treasury bill rate: `tbl_macro`
- Term spread: `tms_macro`
- Default yield spread: `dfy_macro`
- Stock variance: `svar_macro`

Macro variable names are suffixed with `_macro` to avoid conflicts with firm-characteristic names such as `bm`. If the stock panel extends beyond the macro file’s final month, the macro variables are forward-filled to avoid silently dropping the final panel months.

4.4 Characteristic–Macro Interactions

Following the GKX-style specification, the final feature set includes firm characteristics, macro predictors, and characteristic-by-macro interactions. For each characteristic c and macro predictor m , the interaction is:

$$z_{i,t}^{c,m} = x_{i,t}^c \times q_t^m.$$

Interaction columns are named using the pattern:

`characteristic__x__macro`

The final panel is saved as:

`data/processed/panel_1971_2025.parquet`

The feature-group metadata are saved as:

`data/processed/char_cols.csv`

`data/processed/macro_cols.csv`

`data/processed/interaction_cols.csv`

5 Model Training

5.1 Rolling Out-of-Sample Design

The training scripts use an expanding-window out-of-sample design. For each test year y :

- Training sample: all years $\leq y - 3$.
- Validation sample: years $y - 2$ and $y - 1$.
- Test sample: year y .

The first out-of-sample test year is 2001. This corresponds to using the earlier part of the sample for initial training and validation. The final test year is the maximum year available in the panel. In the implemented data, the final usable test period ends in 2024.

A firm-level random subsample is used for computational feasibility. The subsample is selected once by `permno`, so each retained firm enters with its entire history. This avoids introducing look-ahead bias through row-level sampling. The same subsample seed is used in the simple-model and machine-learning scripts so that all models are trained and evaluated on the same stock universe.

5.2 Simple Models

The script `02_train_simple_models.py` estimates four models:

- **OLS+H**: linear model with Huber loss.
- **OLS-3+H**: Huber linear model using size, book-to-market, momentum, macro variables, and their interactions.
- **PCR**: principal-component regression.
- **ENet+H**: elastic net with Huber loss.

OLS+H and ENet+H are implemented using `SGDRegressor` with Huber loss. OLS-3+H is implemented using `HuberRegressor`. PCR standardizes the features, extracts principal components, and estimates a ridge regression on the selected components. The number of principal components is selected using validation mean squared error.

The script saves:

```
outputs/predictions_simple.parquet
outputs/table1_simple.csv
```

5.3 Machine-Learning Models

The script `03_train_ml_models.py` estimates three machine-learning models:

- **RF**: random forest.
- **NN2**: neural network with two hidden layers.
- **NN4**: neural network with four hidden layers.

The random forest is tuned over a small grid of maximum depth and minimum leaf size using validation MSE. The neural networks use a geometric hidden-layer structure and are trained with the Adam optimizer. The neural network regularization parameter is selected using a validation set, and predictions are averaged across multiple random seeds to stabilize performance.

The machine-learning script includes checkpointing. Per-year prediction files are saved to:

outputs/_ckpt_ml/

This allows the script to resume after interruption without rerunning completed test years.

The script saves:

outputs/predictions_ml.parquet

outputs/table1_ml.csv

6 Table 1 Construction

The script `06.make_final1.table.py` combines the simple-model and machine-learning prediction files:

outputs/predictions_simple.parquet

outputs/predictions_ml.parquet

It computes stock-level out-of-sample prediction performance for the seven required models:

OLS+H, OLS-3+H, PCR, ENet+H, RF, NN2, NN4

The main performance metric is:

$$R_{OOS}^2 = 1 - \frac{\sum_{i,t} (r_{i,t+1} - \hat{r}_{i,t+1})^2}{\sum_{i,t} r_{i,t+1}^2}.$$

The benchmark is a zero-return forecast. The script also computes mean and median monthly OOS R^2 .

Two sets are produced:

- Set 1: 1971–2016 sample, evaluated over 2001–2016.
- Set 2: 1971–2024 sample, evaluated over 2001–2024.

The script saves both raw and display-formatted tables:

outputs/table1_final_set1.csv

outputs/table1_final_set2.csv

outputs/table1_final_display_set1.csv

outputs/table1_final_display_set2.csv

It also saves Table 1 figures:

outputs/figure_table1_oos_r2_set1.png

outputs/figure_table1_oos_r2_set2.png

outputs/figure_table1_monthly_r2_set1.png

outputs/figure_table1_monthly_r2_set2.png

7 Figure 4: Variable Importance

The script `04_make_figure4.py` constructs Figure 4 variable-importance heatmaps for the required models:

PCR, ENet+H, RF, NN2, NN4

The importance method follows the GKX intuition of setting one characteristic, and all of its macro interaction terms, to zero. Since characteristics are rank-normalized to $[-1, 1]$ and zero represents the cross-sectional median, this corresponds to replacing a characteristic with its median value. The model is then re-evaluated, and the drop in R^2 is recorded as the variable's importance.

Negative drops are clipped to zero, and importances are normalized to sum to one within each model, making columns comparable across models.

For computational feasibility, the implementation fits one model per algorithm and per sample window rather than recomputing importance across all rolling windows. Importance is measured over a capped random subsample of the estimation window. This means that the magnitudes should be interpreted as approximate model-reliance shares, while the ranking of important variables is the main object of interest.

The script produces:

```
outputs/figure4_importance_set1.png
outputs/figure4_importance_set2.png
outputs/figure4_importance_set1.csv
outputs/figure4_importance_set2.csv
```

8 Figure 9: Long-Short Portfolio Returns

The script `05_make_figure9.py` constructs cumulative returns of machine-learning long-short portfolios. For each model and month:

1. Stocks are sorted into deciles by predicted return.
2. The portfolio goes long the top predicted-return decile.
3. The portfolio goes short the bottom predicted-return decile.
4. Returns are value-weighted within each leg using market equity.
5. The long-short return is the top-decile return minus the bottom-decile return.

The required models for Figure 9 are:

OLS-3+H, PCR, ENet+H, RF, NN2, NN4

The benchmark is S&P 500 minus the risk-free rate. The script first attempts to download the benchmark from WRDS using `crsp.msi` for the S&P 500 monthly return and `ff.factors_monthly` for the risk-free rate. If WRDS is unavailable, the script can use a local fallback file:

`data/benchmark.csv`

The figure also shades NBER recession periods.

The script produces:

`outputs/figure9_cumret_set1.png`
`outputs/figure9_cumret_set2.png`
`outputs/figure9_lsret_set1.csv.csv`
`outputs/figure9_lsret_set2.csv.csv`

9 Outputs

The main output files are saved under `outputs/`. The final report uses the following files:

- `table1_final_display_set1.csv`: Table 1, 1971–2016 sample.
- `table1_final_display_set2.csv`: Table 1, 1971–2024 sample.
- `figure_table1_oos_r2_set1.png`: Table 1 figure, 1971–2016 sample.
- `figure_table1_oos_r2_set2.png`: Table 1 figure, 1971–2024 sample.
- `figure4_importance_set1.png`: Figure 4, 1971–2016 sample.
- `figure4_importance_set2.png`: Figure 4, 1971–2024 sample.
- `figure9_cumret_set1.png`: Figure 9, 2001–2016 out-of-sample period.
- `figure9_cumret_set2.png`: Figure 9, 2001–2024 out-of-sample period.

Large intermediate files are generated locally but are excluded from GitHub:

- `data/processed/panel_1971_2025.parquet`
- `outputs/predictions_simple.parquet`
- `outputs/predictions_ml.parquet`
- `outputs/predictions_all_set1.parquet`
- `outputs/predictions_all_set2.parquet`
- `outputs/_ckpt_ml/`

These files are excluded because they are large and may contain WRDS-derived data.

10 Dependencies

The project uses Python 3.11. The main dependencies are:

- `numpy`
- `pandas`
- `scikit-learn`
- `matplotlib`
- `pyarrow`
- `openpyxl`
- `wrds`
- `openassetpricing`

The recommended installation is:

```
conda create -n gkx python=3.11 -y
conda activate gkx
pip install -r requirements.txt
```

11 Reproducibility Notes

The repository is designed to be reproducible for users with WRDS access. The full pipeline can be executed with:

```
python main.py
```

However, because WRDS-derived data cannot be redistributed, users must download the data themselves by running `01_build_panel.py`. The macro predictor file must also be placed manually in `data/raw/`. Once the panel is constructed, downstream scripts reuse saved intermediate files to avoid redundant computation.

For computational feasibility, the implemented pipeline uses a firm-level random subsample. The subsample is selected at the `permno` level using a fixed random seed, so the same firms are used across simple models and machine-learning models. This maintains consistent model comparisons while reducing runtime on a standard laptop.